

GraphCA: Learning From Graph Counterfactual Augmentation for Knowledge Tracing

Xinhua Wang, Shasha Zhao, Lei Guo, Lei Zhu, Chaoran Cui, and Liancheng Xu

Abstract—With the popularity of online learning in educational settings, knowledge tracing (KT) plays an increasingly significant role. The task of KT is to help students learn more effectively by predicting their next mastery of knowledge based on their historical exercise sequences. Nowadays, many related works have emerged in this field, such as Bayesian knowledge tracing and deep knowledge tracing methods. Despite the progress that has been made in KT, existing techniques still have the following limitations: 1) Previous studies address KT by only exploring the observational sparsity data distribution, and the counterfactual data distribution has been largely ignored. 2) Current works designed for KT only consider either the entity relationships between questions and concepts, or the relations among students, questions, and concepts, simultaneously, leading to inaccurate student modeling. To address the above limitations, we propose a graph counterfactual augmentation method for knowledge tracing. Concretely, to consider the multiple relationships among different entities, we first uniform students, questions, and concepts in graphs, and then leverage a heterogeneous graph convolutional network to conduct representation learning. To model the counterfactual world, we conduct counterfactual transformations on students' learning graphs by changing the corresponding treatments and then exploit the counterfactual outcomes in a contrastive learning framework. We conduct extensive experiments on three real-world datasets, and the experimental results demonstrate the superiority of our proposed GraphCA method compared with several state-of-the-art baselines.

Index Terms—Contrastive learning, counterfactual representation, graph neural network, knowledge tracing.

I. INTRODUCTION

THE development of computer science and artificial intelligence has promoted the progress of the education industry. Many computer-assisted education applications have been

developed rapidly, especially, large-scale online education. More and more researchers are paying more attention to developing learning strategies to help improve their learning effect and efficiency in online learning [1], [2]. Though a large scale of students' learning data can be collected, conducting effective personalized knowledge tracing (KT) in the online scenario is a challenging task due to its inherent data sparsity issue. That is, most students solely answered a small number of questions in history, leading to inaccurate student and question representation learning for KT.

Recently, several studies have been focused on addressing the data sparsity issue of KT. For example, Pandey and Karypis [3] focus on solving the data sparsity problem of KT by only leveraging the students' past activities that are relevant to the given knowledge concept (KC), where a self-attention based approach is proposed. Wang *et al.* [4] propose a novel deep hierarchical knowledge tracing (DHKT) model, which solves the data sparsity problem by using the hierarchical information between questions and concepts to obtain only the rich relationships between questions and concepts. Yang *et al.* [5] utilize the graph convolutional network (GCN) to capture the inter-question and inter-concept abundant relations and propose a graph-based knowledge tracing interaction model to mitigate the data sparsity in KT. Chen *et al.* [6] provide prerequisite relationships between concepts for KT by incorporating the knowledge structure information and creating links between questions for dealing with the data sparsity problem.

However, the data sparsity issue in KT is still unsolved, due to the following limitations of existing methods: 1) Existing methods for solving the data sparsity issue omit the usage of counterfactual data. Counterfactual data refers to the interactions that have not happened, but could, would, or might under differing conditions [7]. Following its definition in causal inference, we can further deem this kind of data as an augmentation of the observed data, providing us with an efficient way to confront the data sparsity issue. The relations between the observed data and counterfactual data are shown in Fig. 1(b). The motivation we leverage the counterfactual technique is that it provides us with an effective way to model the counterfactual data, which further helps us alleviate the data sparsity issue in KT modeling. Counterfactual data is a kind of interaction behavior that is not observational in the past, as one student can only interact (the observed) with one question at the same time. Although the counterfactual data is not observational, it may contain useful information. For example, if we know two questions examine similar knowledge points, and the answer for one of them has been

Manuscript received December 13, 2022; revised March 2, 2023 and April 5, 2023; accepted May 15, 2023. This work was supported by the Natural Science Foundation of China (62372277), the Natural Science Foundation of Shandong Province (ZR2022MF257, ZR2022MF295) and Humanities and Social Sciences Fund of the Ministry of Education (21YJC630157). Recommended by Associate Editor Wenguan Wang. (Corresponding author: Lei Guo.)

Citation: X. H. Wang, S. S. Zhao, L. Guo, L. Zhu, C. R. Cui, and L. C. Xu, "GraphCA: Learning from graph counterfactual augmentation for knowledge tracing," *IEEE/CAA J. Autom. Sinica*, vol. 10, no. 11, pp. 2108–2123, Nov. 2023.

X. H. Wang, S. S. Zhao, L. Guo, L. Zhu, and L. C. Xu are with the School of Information Science and Engineering, Shandong Normal University, Jinan 250358, China (e-mail: wangxinhua@sdu.edu.cn; zhaoshasha2242@163.com; guolei@sdu.edu.cn; leizhu0608@gmail.com; lcxu@sdu.edu.cn).

C. R. Cui is with the School of Computer Science and Technology, Shandong University of Finance and Economics, Jinan 250014, China (e-mail: crcui@sdufe.edu.cn).

Color versions of one or more of the figures in this paper are available online at <http://ieeexplore.ieee.org>.

Digital Object Identifier 10.1109/JAS.2023.123678

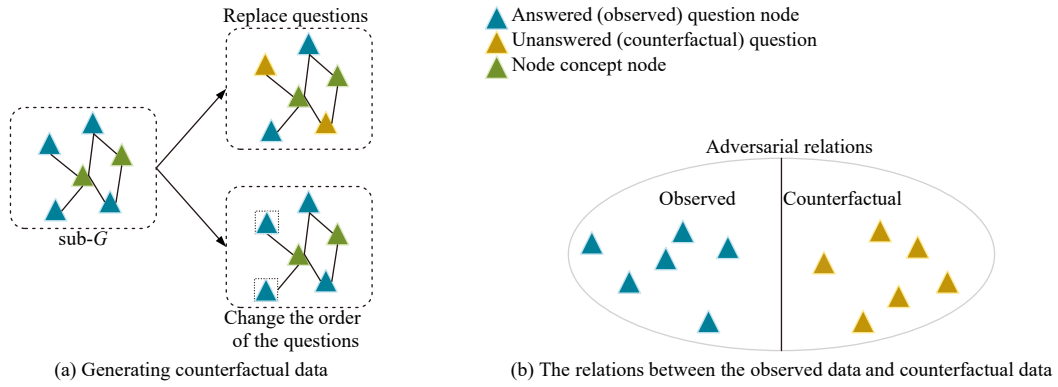


Fig. 1. An illustration of modeling unobserved data from a counterfactual perspective.

recorded, then, we can deduce similar answers to the other one. By doing this, we can augment the answer records of students to alleviate the data sparsity issue. Intuitively, modeling the counterfactual world enables us to consider more unobserved characteristics of students, and thus can learn more accurate representations for both students and questions. Motivated by the work in [7], we generate the counterfactual data for every student according to the following strategies (as we have shown in Fig. 1(a), and more details can be seen in Section III-D): The question replacement operation and counterfactual disruption. The example of the resulting sub-graph is shown in Fig. 1(a). Counterfactual transformation can be seen as one kind of data augmentation method. 2) Another limitation of existing works is that existing studies fail to capture the structure information among multiple entities, i.e., the relationships among students, questions, and concepts. And, as the students can only interact with a small number of questions (data is sparse), directly modeling them is infeasible. One intuitive way of dealing with this issue is to resort to other relationships among different entities. However, previous works only consider either the entity relationships between questions and concepts, or the relations between two concepts, and none of them investigates their relationships, simultaneously. To overcome this challenge, we further consider the student-question relationship in a unified graph. We incorporate students into the graph to further consider the student-question-concept relationship mainly because students' knowledge state can be modeled by the questions that they have answered. Though some questions may have the same concepts, we can still distinguish the students by their different answers to these questions, as we present students not only by the questions, but also by their answers to the questions. The more information involved with a student we have, the more likely we are to better modeling students. As the direct connection with students only involves questions, incorporating concepts would not increase the combinatorial space. Instead, further considering the concepts, enables us to learn more accurate question representations, which will further affect students' representation learning. Modeling students, questions, and concepts in a unified graph facilitates us in learning their high-order relationships. An example of modeling them in a unified graph is shown in Fig. 2.

To overcome the above limitations, we propose a counterfactual graph counterfactual augmentation method for knowl-

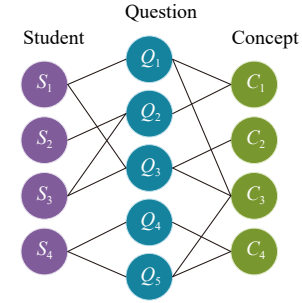


Fig. 2. An example of modeling the relationships among students, questions, and concepts in a unified graph.

edge tracing, namely GraphCA. Our main idea to confront the data sparsity problem is to model the counterfactual data distribution rather than solely the observational data distribution. Specifically, we aim to answer the following counterfactual question, “what the student representation would be if we intervene on the observed answers”. More concretely, given the observed question sequence, we first unify students, questions, and concepts in a student-question-concept (SQC) relation graph to model their higher-order relationships. Then, we intervene on the observed answers by generating interrupted sub-graphs, from which we can obtain counterfactual student representation via counterfactual contrastive graph learning. In counterfactual learning, we perform counterfactual graph transformations from two views based on two factual observations. First, the student's knowledge state represented by the question sequence may remain similar even if we replace them with similar questions. Motivated by this, we obtain counterfactually positive user representation by randomly replacing similar questions within the question sequence. Second, the student's knowledge state is independent of the question orders that the student has answered. As such, we achieve the counterfactually positive samples by randomly changing the order of the questions. The goal of our counterfactual strategies is to answer two “what if...” questions. The first strategy that intervenes with the students by the question replacement operation tends to answer “what the student representation would be if we intervene on the observed answers”. The second strategy that disrupts students' answer records by changing their answer orders tends to answer “what the student representation would be if...”. We resort to

the counterfactual learning technique to generate the augmented data mainly because it provides us with an efficient way to explore the unobserved data, which was seldom investigated in previous works. Another reason we leverage counterfactual learning as our solution is that the augmented data can help us to answer the “what if...” questions under the potential outcome framework, which cannot be guaranteed by other related methods. In both cases, we reach the negative representation by directly corrupting the student’s original representation (the details can be seen in Section III-E-3). Then, to effectively learn a more stable user representation, we further conduct contrastive learning between the observational and the counterfactual user representations. That is, by contrast with such out-of-distribution hard negative samples, the learned representations are potentially more stable, since they are less sensitive to noisy questions. Moreover, contrasting with out-of-distribution positive samples potentially makes the learned representations more accurate, since they trust the indispensable samples more.

The main contributions of this work are summarized as follows:

1) We focus on the data sparsity issue of knowledge tracing and solve it by leveraging the counterfactual data in an innovatively devised counterfactual contrasting graph learning method, namely GraphCA. More concretely, we first obtain the counterfactual positive samples by generating interrupted sub-graphs based on two observational facts, and then learn an enhanced user representation by a contrastive graph learning method.

2) We consider the multiple relationships among students, questions, and concepts in a unified heterogeneous graph to enhance the representations of students by the concepts involved in questions.

3) We conduct extensive experiments on three real-world datasets, and the experimental results demonstrate the superiority of our method compared with several state-of-the-art baselines.

II. RELATED WORK

In this section, we briefly review existing works on traditional KT, graph-based KT, contrastive learning-based KT, the studies that solve the data sparsity issue in KT, and Heterogeneous graph neural networks. Then, we review and discuss the counterfactual learning that is related to our research.

A. Traditional KT Methods

Traditional KT methods can be classified into three categories, i.e., cognitive diagnosis models (CDMs), Bayesian-based methods, and RNN-based methods.

1) *Cognitive Diagnosis Model*: Cognitive diagnosis model (CDM) is a technique that classifies learners’ mastery of knowledge concepts through their answer records [8], which can be further divided into two categories, i.e., continuous and discrete modeling methods [9]. As for continuous models, Deonovic *et al.* [10] propose an item response theory (IRT)-based method to describe the students’ comprehensive knowledge state. However, IRT-based methods only consider the static proficiency value of students, which is insufficient for

tracing students’ dynamic knowledge. As for discrete models, De La Torre [11] introduce a Q matrix (question-concept matrix) with each dimension representing the student’s mastery of the knowledge concept to model Deterministic Inputs. But this work suggests that students’ abilities are static, which is inconsistent with the reality that students’ abilities are dynamic and changing.

2) *Bayesian-Based Methods*: In this category, Bayesian knowledge tracing (BKT) [12] is the first work that investigates knowledge tracing by devising a BKT-based method, where a hidden Markov model (HMM) is utilized to model the learner’s knowledge mastery level and a set of binary variables to model student’s knowledge states. In previous studies on BKT [13], [14], a common approach is to introduce learning parameters such as the probability of student guesses and errors, the difficulty of the question, the ability of the student, and the learning rate of the learner, to learn more information about the student’s learning process. However, the disadvantage of the BKT-based approach is that it assumes that once a student learns a concept, he/she will never forget it [15]. But in fact, students’ learning process is usually accompanied by severe knowledge forgetting.

3) *RNN-Based Methods*: Existing methods in this category often tackle KT as a sequential modeling task [16]–[18]. For example, Piech *et al.* [18] propose a deep learning-based knowledge tracing (DKT) model by using a recurrent neural network (RNN) to track students’ knowledge states. Zhang *et al.* [19] propose a dynamic key-value memory network (DKVMN) model to study students’ mastery level of each concept. Nagatani *et al.* [20] further add students forgetting information based on DKT to enhance the performance prediction for students. Abdelrahman and Wang [21] propose sequential key-value memory networks by unifying the recurrent modeling capability of DKT and the memory capability of DKVMN to model student learning. Liu *et al.* [22] take into account the student’s exercising records and the text content of corresponding exercises by a bidirectional long short-term memory (LSTM) to learn student representations. However, this kind of method mainly focused on modeling students’ learning sequences, and the high-order relationships between questions and concepts are ignored.

B. Graph-Based KT Methods

To model the high-order relationships between questions and concepts, researchers have leveraged the power of graph neural network for KT [23]–[26], which have achieved great success in many areas, such as recommender systems [27], computer vision [28], and natural language processing [29]. For example, Liu *et al.* [30] propose a graph-based pre-training approach by considering question-question, question-concept, and concept-concept relationships in a bipartite graph to enhance the representations of questions. Nakagawa *et al.* [31] propose a GNN-based knowledge tracing method, which defines the knowledge structure as a graph to transform the KT task into a time-series node-level classification problem in the GNN. Yang *et al.* [5] propose a graph-based interaction model for knowledge tracing model by using GCN to incorporate the relationship between questions and concepts while

considering the historical state of the whole exercise. Tong *et al.* [32] utilize multiple relations of knowledge structures to explore the relationships between concepts and consider both temporal and spatial effects to achieve modeling transfer of knowledge. However, graph-based approaches only consider the entity relationships between questions and concepts, or between two concepts, and none of them investigates the relations among students, questions, and concepts, simultaneously.

C. Contrastive Learning-Based KT Methods

Contrastive learning is a type of self-supervised learning method that aims at learning more robust and accurate representations by pulling similar representations closer and pushing away dissimilar representations. Contrastive learning has achieved significant improvements over traditional methods in the area of natural language processing [33], [34], computer vision [35], [36], and recommender systems [37], [38]. For example, Hassani and Khasahmadi [39] introduce a self-supervised approach by comparing two structural views of the graph to learn node and graph level representations. Hafidi *et al.* [40] propose a graph contrastive learning framework by maximizing the similarity between the representations of two mapping methods for the same node to learn node embeddings. Recently, several works have also been focused on developing contrastive learning-based solutions for KT. For example, Song *et al.* [41] propose a contrastive learning framework to obtain the effective representation of knowledge states and improve the ability to predict learners' performance. Lee *et al.* [42] use multiple transformers to encode students' learning history and design bidirectional encoders within a contrastive learning framework to reflect useful self-supervised signals of historical features. However, none of the existing methods solves KT by contrastive learning from a counterfactual learning view, and its effectiveness needs to be further explored.

D. Methods Solving Data Sparsity in KT

We notice that there are some related works that have been focused on solving the data sparsity issue in KT. For example, Chen *et al.* [6] deal with the data sparsity problem by incorporating knowledge structure to obtain prerequisite relationships between concepts. Pandey and Kargpis [3] identify the knowledge concepts from the student's past activities by a self-attention network. Wang *et al.* [4] solve the data sparsity problem by using the hierarchical information between questions and concepts and only obtaining the rich relationships between questions and concepts. Yang *et al.* [5] mitigate the data sparsity problem by defining the relationship between questions and concepts on the graph to explore the higher-order information. Lee *et al.* [42] use contrastive learning to conduct knowledge tracing and propose four data augmentation strategies (CL4KT) to model students' knowledge states. But their strategies are different from ours in the following aspects: 1) We conduct the counterfactual augmentations on the basis of graphs, while CL4KT only deals with sequential data, where the connections among the entities are ignored. 2) We devise a new augmentation strategy to generate positive sam-

ples, that is, we define the positive samples by randomly replacing the questions that are similar to existing ones. However, all the existing methods solve the data sparsity issue by only modeling the distributions of observational data, while the unobserved counterfactual data is not leveraged. In light of this, in this work we propose to solve the data sparsity problem by modeling the counterfactual data distribution from a counterfactual perspective rather than solely the observational data distribution.

E. Heterogeneous Graph Neural Networks.

Heterogeneous graph neural network (HGNN) is a kind of graph network that models different types of nodes and links [43], [44]. Compared with traditional graphs, hierarchical graphs are more capable of expressing complex and hierarchical relationships. Existing studies on HGNN are mainly focused on link prediction [45], [46], personalized recommendation [47], [48], and node classification [49]. For example, Xu *et al.* [45] propose a topic-aware heterogeneous graph neural network (THGNN) to hierarchically mine topic-aware semantics for learning link prediction of multifaceted node representations. Shi *et al.* [47] devise a heterogeneous graph neural network for recommendation (HGRec), which obtains node embeddings by aggregating neighbors based on multi-hop meta-paths and adding attention mechanisms to fuse rich semantics. Zhao *et al.* [49] develop a heterogeneous graph structure learning (HGSL) framework, which jointly learns heterogeneous graph and GNN parameters to improve node classification performance. However, none of the above studies pays attention to its power of modeling students' knowledge state. Inspired by the capability of HGNN in modeling the relationships among different types of entities, we further introduce it to the KT task to learn modeling students, questions, concepts, and their corresponding relationships.

F. Counterfactual Learning

Counterfactuals are a process by which humans substitute unreal conditions or possibilities. A typical counterfactual problem can be described as: "what... would be if...?". In previous studies [7], [50]–[52], counterfactual learning is usually exploited to address the problem of fairness or improve the robustness of their models. For example, Kusner *et al.* [51] argue that it is critical to correctly address the causality of fairness by describing the process of designing fairness algorithms with causal inference techniques. Zhang *et al.* [7] model counterfactual data from item level and interest level to learn accurate and robust user representations. Besides the above applications, counterfactual learning has also been used to address the data sparsity problem in natural language processing (NLP) [53], computer vision (CV) [54], and cross-modal (e.g., visual-verbal) [55]–[57] areas, where counterfactual learning is utilized to provide the data augmentations for modeling training. Abbasnejad *et al.* [55] propose a counterfactual framework to enrich the data by generating a set of counterfactual examples for addressing the problem of data spurious correlation. Chen *et al.* [56] propose a model-independent counterfactual sample synthesis training method, which has generated a large number of counterfactual training

samples by considering two different synthesis mechanisms of masking critical objects or words. Wang *et al.* [57] propose the counterfactual cycle-consistent (CCC) framework, which applies counterfactual learning to cross-tasks in order to learn instruction following and generation simultaneously. The data augmentation methods of graph structure are generally categorized as node dropping, edge perturbation, attribute masking, and subgraph-based methods [58]–[60]. We notice that there are also studies conducting graph data augmentations by counterfactual learning. For example, Zhao *et al.* [61] develop a data augmentation-based link prediction method that creates counterfactual links and learns representations from observed and counterfactual links. But, their work is proposed for link prediction, and the advantage of counterfactual graph data augmentation on KT modeling is still unexplored. In our setting, we follow the definition in [62] proposed by Imbens and Rubin. Then, we can define our counterfactual transformation as one of the data augmentation methods that generate counterfactual data from the unobserved world following the data transformation strategies, by which we can answer the “what... if...”. Moreover, their work does not consider the robustness of the learned representations, which is further investigated in our proposed method by combining it with contrastive learning.

III. METHODOLOGIES

This section first introduces the definitions used in this work, then overviews our proposed KT method, and finally describes each of its components in detail.

A. Preliminaries

Definition 1: The knowledge tracing task: Suppose we know a student’s historical exercise interaction sequence $s_N = \{(q_1, a_1), \dots, (q_N, a_N)\}$, where q_i is the i -th question that the student has answered, and $a_i \in \{0, 1\}$ represents the correctness of the student’s answer to q_i . Let $a_i = 1$ if the answer is correct and $a_i = 0$, otherwise. Given the sequence of interactions s_N and the next question q_{N+1} , the task of KT is to predict the probability that the student will correctly answer q_{N+1} , with probability $p(a_{N+1} = 1 | s_N, q_{N+1})$.

Definition 2: Student-question-concept (SQC) graph: Let $S = \{s_1, s_2, \dots, s_j, \dots, s_n\}$, $Q = \{q_1, q_2, \dots, q_h, \dots, q_m\}$ and $C = \{c_1, c_2, \dots, c_l, \dots, c_k\}$ be the sets of students, questions and concepts, where m , n and k denote the numbers of questions, students and concepts, respectively. Regarding the relationship between students and questions, we suppose that one student can answer multiple questions, and a question can also be answered by more than one student. For questions and concepts, we assume that one question contains one or more concepts, and one concept is covered by multiple questions, i.e., there is a many-to-many relationship between questions and concepts. As thus, we can conclude two kinds of interactions between S , Q , and C , that is, student-question interactions and question-concept interactions, which are respectively denoted by $\mathbf{R}^s = [r_{jh}^s]^{n \times m}$ and $\mathbf{R}^q = [r_{hl}^q]^{m \times k}$, where $r = 1$ indicates the observed interactions and $r = 0$ denotes the unobserved ones.

Definition 3: Counterfactual learning: Let X be the set of contexts, T be the set of treatments, and Y be the set of outcome values. For a context $x \in X$ and a treatment $t \in T$, we

denote the outcome of x under the treatment t by $Y_t(x) \in Y$. Ideally, we would need all possible outcomes of x under all treatments to learn the causal relationships. However, in reality, only one treatment is applied and thus only one outcome is obtained when a content x is given.

B. Overview of GraphCA

In this work, we tend to answer the following counterfactual question, “what the student representation would be if we intervene on the observed answers”. As we know, conducting counterfactual learning usually has to explore the following three factors: context, treatment, and outcome, where the context is the given background variable; the treatment is the independent variable, which is randomized; and the outcome is the dependent variable, which is related to treatment. In our method, we treat the student node as the context X , whether the student’s answering records have been intervened as the treatment T , and the student’s representation as the outcome Y . To answer “what the student representation would be”, we construct the augmentation views on the basis of two factual observations. Our counterfactual transformation strategies are developed based on existing causal inference theory, and they also follow the SUTVA [62] assumption. The overall architecture of our GraphCA is shown in Fig. 3. Specifically, we first find the questions (from unanswered questions) that are similar to the ones she has answered. Then, we disturb her historical answers by the following two strategies: 1) We randomly replace the questions in her historical answer record with similarly unanswered questions to generate the unobserved answer sequence. As we replace the questions with similar ones, we deem the disturbed sequence is similar to the original one. 2) Another way to obtain unobserved student answer records is to disrupt the answer sequence of the same question set. As the disturbed answer record has the same question set, we also deem this answer sequence has a similar meaning to the original one. After that, we achieve students’ counterfactual representations by performing a graph convolution layer on the transformed graph, where we treat the counterfactual representations from the above generated answer sequences as positive samples, and their representation transformation forms as negative samples (the details can be seen in Section III-E-3). Then, we further conduct contrastive learning on the original and counterfactual representations to model students more accurately and robustly.

C. SQC Graph Construction

To consider the hierarchical relationships among students, questions, and concepts, we first model them in a unified graph (i.e., the SQC graph), which further serves as the foundation of the counterfactual graph transformation and student representation learning. Specifically, we define $G = \{(s, r^s, q, r^q, c) \mid s \in S, q \in Q, c \in C\}$ as the constructed graph that models the relationship among students, questions, and concepts, where s , q , and c denote the student s , the question q , and the concept c , respectively. r^s and r^q indicate the relationships between the student-question and the question-concept, respectively. In SQC, there are three types of nodes, i.e., students, questions, and concepts, and two kinds of edges, i.e.,

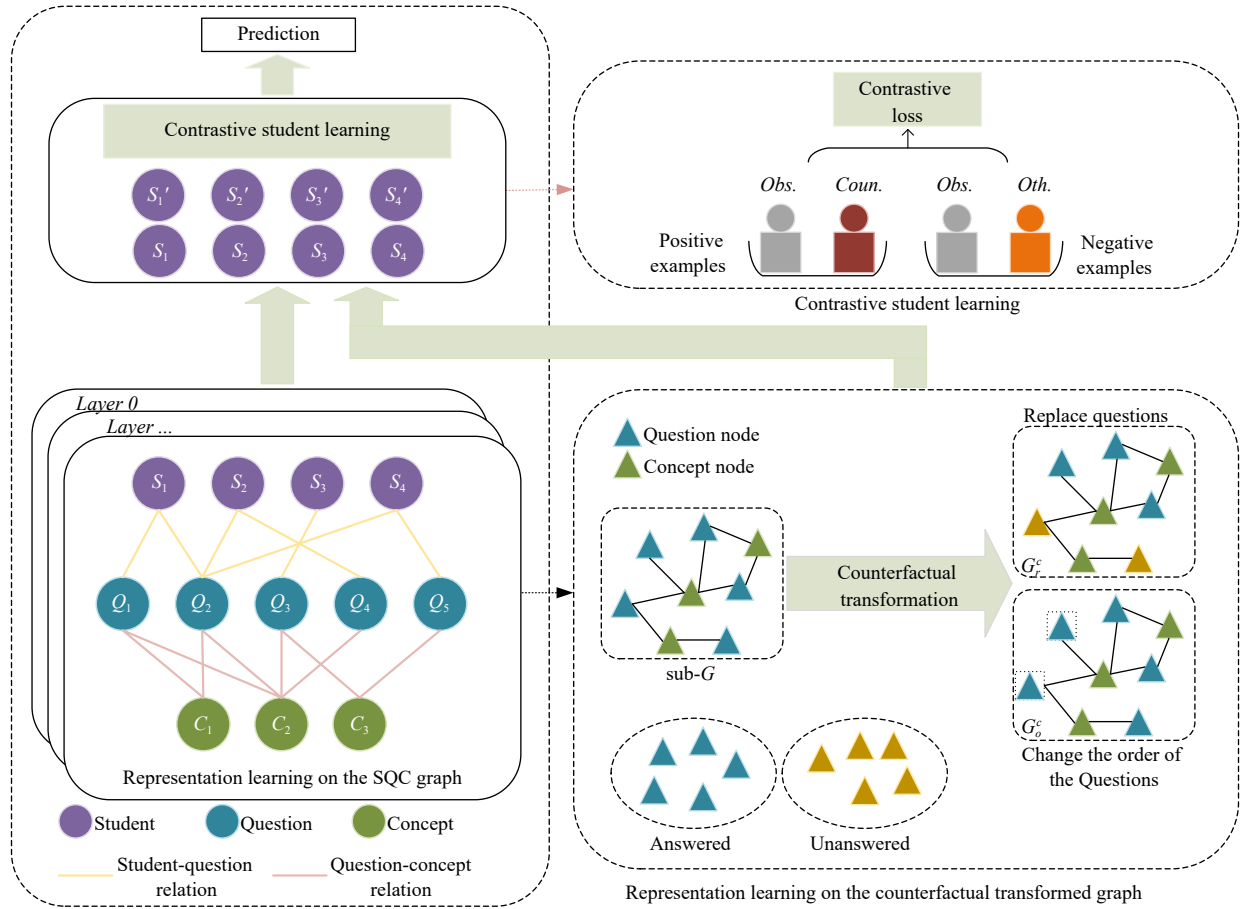


Fig. 3. Overview of the architecture of GraphCA.

the relationships between students and questions, and the relationships between questions and concepts. For student s_j , question q_h , there is a connection between s_j and q_h only if the student interacts with the question. For question q_h and concept c_l , if q_h contains c_l , we build a connection between q_h and c_l to indicate their question-concept relation.

D. Graph Data Augmentation

To alleviate the inherent data sparsity issue in the knowledge tracing task, we resort to the modeling of the counterfactual world, which not only contains observed data but also unobserved data. To provide data foundations for counterfactual learning, we further conduct graph transformations to augment the observed data by the following two transformation strategies.

1) *Counterfactual Replacement With Similar Questions:* In this work, for each student node (context), the observed data contains only the treatment with true values, and the corresponding outcome of the fact. To answer “what the student representation would be if we intervene on the observed answers”, and obtain the outcome of each student node (context) under different treatments, we intervene with the students by the question replacement operation [63]. Moreover, since the answers of each student to the unanswered questions are unknown, we use the result of the nearest observed question as a substitute. Here, the nearest neighbor question refers to the nodes with common concepts in the SQC graph. To

construct the counterfactual sub-graphs denoting students’ answer sequences, which have similar potential outcomes as the corresponding factual observations, we conduct graph augmentations by the following replacement strategy: 1) We first identify the unanswered questions that are similar to the ones that have been answered by the students. 2) Then, we perform graph augmentation on the original sub-graph $sub-G$ for each student by randomly replacing the question nodes with the most similar questions as their substitutes (from the sets of unanswered questions). We obtain the nearest node q_k to the replaced question node q_i according to the following method:

$$q_k = \arg \min_{q_i, q_k \in Q} \{d(q_i, q_k)\} \quad (1)$$

where $d(\cdot, \cdot)$ is used to measure the distance between two nodes. We use cosine similarity to calculate the similarity between two questions, which can express the degree of preference between nodes and learn the degree of similarity between nodes. Its definition is shown as follows:

$$sim_{ik} = \frac{\mathbf{q}_i \mathbf{q}_k}{\|\mathbf{q}_i\|_2 \|\mathbf{q}_k\|_2} \quad (2)$$

where $\mathbf{q}_i$ and $\mathbf{q}_k$ are the embeddings of questions q_i and q_k , respectively. As we replace the questions with similar ones, we deem that the disturbed sequence is similar to the original one and the student representations we obtained through GCN are similar to the observed student representations. When performing counterfactual replacement, the number of replaced

questions M is set as a hyper-parameter and we fine-tune it in Section VI-B. Note that, if the length of the answer sequence is less than M , then we replace the question sequence with the same number of answers.

2) *Counterfactual Disruption*: The counterfactual replacement strategy tends to obtain the unobserved student answer records by replacing already answered questions. But this method also introduces some unanswered questions, which brings uncertainty to our approach. To overcome this challenge, another intuitive way is to disrupt students' answer records by changing their answer orders, which have the same question set before and after the disturbance. Intuitively, the student's knowledge state obtained from the same question set should also be similar. More formally, we answer the "what if..." question by finding the potential outcome of each student node on the counterfactual fact that the student answered these questions in different orders, where the treatment is denoted as whether we have changed the students' answer orders. Then, how to generate the counterfactual data by changing the students' answer orders is one of our main tasks. That is, we tend to change the order of student's answers, so that we can obtain the outcome of the student node (context) under different treatments.

Specifically, we conduct the following disruption strategy on the sub-graph: for the same student node (context), we randomly select two question nodes connected with it, and then exchange them on the sub-graph with their relationships remaining fixed. That is, when disrupting the sequence of answers on the sub- G , we only change the question nodes, their relationships with concept nodes and student nodes are unchanged. As the disturbed answer record on the sub- G still has the same question set, we can deem this answer sequence has a similar meaning to the original one, and the student representation obtained through the following graph convolutional network is still similar to the observed student representations (the outcome of the context). In experiments, we treat the number of swap question pairs as a hyper-parameter, whose value is set as the half length of the answer sequence if the length of a student's answer sequence is less than twice the number of swapped question pairs.

E. Inducing Counterfactual Representations With Contrastive Graph Learning

In this work, we leverage the graph convolutional network (GCN) as the backbone of our solution, which provides us with an efficient way to induce the high-dimensional student, question, and concept features from students' learning graphs. Concretely, two kinds of student representations (i.e., outcomes), i.e., the outcomes before and after the treatment is changed, are learned by two GCNs (the workflow of the learning process is shown in Fig. 3).

1) *Inferring Representations From the Original Student Learning Graph*: In the SQC graph, there are three kinds of nodes, i.e., students, questions, and concepts, and two kinds of edges, i.e., the relationships between students and questions, and the relationships between questions and concepts. Let s , q , and c be noted as student nodes, question nodes, and concept nodes, respectively. r^s and r^q indicate the relationships

between the student-question and the question-concept, respectively.

a) *Concept representation learning*: In this work, we represent concepts by the questions that they belong to. For a concept, its representation is learned by aggregating the information from itself and the question nodes connected with it. Formally, we denote the representation of concept node c_m in graph G as c_m and the question set that is directly connected with it as N_m^c . Then, we define the message passed from the question node set N_m^c to c_m in the l -th layer GCN as

$$\text{message}_{c_m \leftarrow N_m^c}^l = \frac{1}{|N_m^c|} \sum_{n \in N_m^c} w^l q_n^{l-1} + b^l \quad (3)$$

where $\text{message}_{c_m \leftarrow N_m^c}^l$ represents the message passed from N_m^c to c_m , w^l and b^l are the weights and biases to be learned, respectively. $|N_m^c|$ is the neighbor set number of c_m . In addition, to retain the information about the concept node itself, we add a self-connection to it. The aggregation strategy is shown as follows:

$$\text{message}_{c_m \leftarrow c_m}^l = \frac{1}{|N_m^c|} w^l c_m^{l-1} + b^l \quad (4)$$

where $\text{message}_{c_m \leftarrow c_m}^l$ represents the message passed from c_m to c_m .

After aggregating the message passed from the concept itself and the question, we use a *ReLU* activation function. The calculation formula of the l -th layer GCN can be defined as

$$c_m^l = \text{ReLU}(\text{message}_{c_m \leftarrow c_m}^l + \text{message}_{c_m \leftarrow N_m^c}^l). \quad (5)$$

b) *Question representation learning*: In the SQC graph, question nodes are connected with concepts that are contained in the question, as well as the students that have answered the current question. Then, for a question, its representation is learned by aggregating the information from itself and the concepts and students connected with it. In addition, the students' answers also have an impact on question representation learning. Hence, we describe the representation of a question by simultaneously considering its answer. We denote the representation of the question q_h as q_h . Let $N_h^{q,c}$ and $N_h^{q,s}$ be concept set and student set that are connected with q_h , respectively. Then, we can define the message passed from the concept nodes $N_h^{q,c}$ to q_h as

$$\text{message}_{q_h \leftarrow N_h^{q,c}}^l = \frac{1}{|N_h^{q,c}|} \sum_{k \in N_h^{q,c}} w^l c_k^{l-1} + b^l \quad (6)$$

where $\text{message}_{q_h \leftarrow N_h^{q,c}}^l$ represents the message passed from $N_h^{q,c}$ to q_h . $|N_h^{q,c}|$ is the number of neighbor concept nodes of q_h . The message passed from the student nodes $N_h^{q,s}$ to q_h can be similarly defined as

$$\text{message}_{q_h \leftarrow N_h^{q,s}}^l = \frac{1}{|N_h^{q,s}|} \sum_{k \in N_h^{q,s}} w^l s_k^{l-1} + b^l \quad (7)$$

where $\text{message}_{q_h \leftarrow N_h^{q,s}}^l$ represents the message passed from $N_h^{q,s}$ to q_h . $|N_h^{q,s}|$ is the number of neighbor student nodes of q_h .

q_h . Similarly, the message retained by the question itself is denoted as

$$\mathbf{message}_{q_h \leftarrow q_h}^l = \frac{1}{|\mathcal{N}_h^q|} \mathbf{w}^l \mathbf{q}_h^{l-1} + \mathbf{b}^l \quad (8)$$

where $\mathbf{message}_{q_h \leftarrow q_h}^l$ represents the message passed from q_h to q_h .

Thereafter, we aggregate the messages passed from the question itself, the concept, and the student, and achieve the representation of question q_h in the l -th layer as

$$\begin{aligned} \mathbf{q}_h^l = & \text{ReLU}(\mathbf{message}_{q_h \leftarrow q_h}^l + \mathbf{message}_{q_h \leftarrow \mathcal{N}_h^{q,c}}^l \\ & + \mathbf{message}_{q_h \leftarrow \mathcal{N}_h^{q,s}}^l) \end{aligned} \quad (9)$$

where ReLU is the none-linear activation function.

c) Student representation learning: Due to students being only connected with the questions that he/she has answered, the representation of a student is learned by aggregating the information from himself/herself and the corresponding questions. Let s_j be the representation of student node s_j , and $\mathcal{N}_j^s$ be the neighbor question set. Then, we can define the passed messages from the question nodes $\mathcal{N}_j^s$ to s_j in the l -th layer as

$$\mathbf{message}_{s_j \leftarrow \mathcal{N}_j^s}^l = \frac{1}{|\mathcal{N}_j^s|} \sum_{i \in \mathcal{N}_j^s} \mathbf{w}^l \mathbf{q}_i^{l-1} + \mathbf{b}^l \quad (10)$$

where $\mathbf{message}_{s_j \leftarrow \mathcal{N}_j^s}^l$ represents the message passed from $\mathcal{N}_j^s$ to s_j . $|\mathcal{N}_j^s|$ is the number of neighbor nodes of s_j . The message retained by the student itself is denoted as

$$\mathbf{message}_{s_j \leftarrow s_j}^l = \frac{1}{|\mathcal{N}_j^s|} \mathbf{w}^l \mathbf{s}_j^{l-1} + \mathbf{b}^l \quad (11)$$

where $\mathbf{message}_{s_j \leftarrow s_j}^l$ represents the message passed from s_j to s_j .

Similarly to concepts and questions, we arrive at the student's final representation by aggregating the messages from himself/herself and the corresponding questions

$$\mathbf{s}_j^l = \text{ReLU}(\mathbf{message}_{s_j \leftarrow s_j}^l + \mathbf{message}_{s_j \leftarrow \mathcal{N}_j^s}^l) \quad (12)$$

where $\mathbf{s}_j^l$ denotes the representation of the student s_j in the l -th layer.

2) Representation Learning on the Transformed Counterfactual Graph: As described in Section III-D, we obtain two augmented sub-graphs G_r^c and G_o^c by performing counterfactual graph transformations to achieve the outcomes of our experiments when the treatment is changed. Correspondingly, we can achieve two types of student representations by the same GCN encoder, which are further treated as the counterfactual outcomes of students' learning states.

a) Student representation learning with counterfactual replacement: As in the transformed counterfactual graph G_1 , student nodes are only connected with the questions that have been answered by the students, we can denote the representation of s_j by aggregating the messages from questions. We let $\mathcal{N}_j^{s,r}$ be neighbor question set. The message is denoted as

$$\mathbf{message}_{s_j \leftarrow \mathcal{N}_j^{s,r}}^l = \frac{1}{|\mathcal{N}_j^{s,r}|} \sum_{i \in \mathcal{N}_j^{s,r}} \mathbf{w}^l \mathbf{q}_i^{l-1} + \mathbf{b}^l \quad (13)$$

where $\mathbf{message}_{s_j \leftarrow \mathcal{N}_j^{s,r}}^l$ represents the message passed from $\mathcal{N}_j^{s,r}$ to s_j . $|\mathcal{N}_j^{s,r}|$ is the number of neighbor nodes of s_j . The message from the student itself is denoted as

$$\mathbf{message}_{s_j \leftarrow s_j}^l = \frac{1}{|\mathcal{N}_j^{s,r}|} \mathbf{w}^l \mathbf{s}_j^{l-1} + \mathbf{b}^l \quad (14)$$

where $\mathbf{message}_{s_j \leftarrow s_j}^l$ represents the message passed from s_j to s_j .

By integrating the above messages, we can define the student representation as follows:

$$\mathbf{s}_j^{l,r} = \text{ReLU}(\mathbf{message}_{s_j \leftarrow s_j}^l + \mathbf{message}_{s_j \leftarrow \mathcal{N}_j^{s,r}}^l). \quad (15)$$

b) Student representation learning with counterfactual disruption: Similarly to the counterfactual graph G_1 , the messages that are passed to student s_j^o are also from himself/herself and the questions that he/she has answered. The message from the question nodes is defined as

$$\mathbf{message}_{s_j \leftarrow \mathcal{N}_j^{s,o}}^l = \frac{1}{|\mathcal{N}_j^{s,o}|} \sum_{i \in \mathcal{N}_j^{s,o}} \mathbf{w}^l \mathbf{q}_i^{l-1} + \mathbf{b}^l \quad (16)$$

where $\mathbf{message}_{s_j \leftarrow \mathcal{N}_j^{s,o}}^l$ represents the message passed from $\mathcal{N}_j^{s,o}$ to s_j . $|\mathcal{N}_j^{s,o}|$ is the number of neighbor nodes of s_j . The message passed from himself/herself can be defined as

$$\mathbf{message}_{s_j \leftarrow s_j}^l = \frac{1}{|\mathcal{N}_j^{s,o}|} \mathbf{w}^l \mathbf{s}_j^{l-1} + \mathbf{b}^l \quad (17)$$

where $\mathbf{message}_{s_j \leftarrow s_j}^l$ represents the message passed from s_j to s_j .

Then, the final representation of student s_j^o can be written as

$$\mathbf{s}_j^{l,o} = \text{ReLU}(\mathbf{message}_{s_j \leftarrow s_j}^l + \mathbf{message}_{s_j \leftarrow \mathcal{N}_j^{s,o}}^l). \quad (18)$$

3) Enhance Student Representation by Contrastive Learning: To enhance the learned student representations, we further incorporate contrastive learning objectives. As we have described in Section III-D, the student representations induced from the counterfactual graphs indicate similar students' learning states both before and after the treatment is changed, we treat both outcomes as positive representations of students. To make a contrast, for both transformation views, we directly generate the corresponding negative student representations by a destruction function $\mathcal{P}$. It is denoted as

$$\tilde{\mathbf{s}}_j^l = \mathcal{P}(\mathbf{s}_j^l) \quad (19)$$

where $\mathbf{s}_j^l$ indicates the originally observed student representation, and $\tilde{\mathbf{s}}_j^l$ denotes the negative student representation, which is obtained by corrupting the observed student representations $\mathbf{s}_j^l$ with row-wise and column-wise shuffling. Consistently with the transformation views, we also have two contrastive learning objects, which are shown as follows:

a) Contrast with the outcome of the counterfactual replace-

ment: We consider the outcome of the first counterfactual transformation as the positive sample, and the destructed representation as the student's negative sample. Then, to minimize the distance between the student's representation and the positive sample, and maximize its distance with the negative samples, we use InfoNCE [64] with a standard binary cross-entropy loss as our learning objective. By doing this, we effectively learn a more accurate and robust representation of the student. Formally, we have

$$\mathcal{L}_{cr} = - \sum_{j=1}^n (\log \sigma(f_D(s_j^l, s_j^{l,r})) + \log \sigma(1 - f_D(s_j^l, \tilde{s}_j^l))) \quad (20)$$

where n denotes the numbers of students, $s_j^{l,r}$ denotes student representation with counterfactual replacement. $f_D(\cdot)$ is the discriminator function that takes two vectors as input and then scores them for consistency. We use the dot product between the two representations to achieve discrimination.

b) Contrast with the outcome of the counterfactual disruption: In this counterfactual disruption view, we also treat the outcome of the counterfactual transformation as positive samples, and students' destruction as negative samples.

By performing InfoNCE loss, we can get the second contrastive learning objective function

$$\mathcal{L}_{co} = - \sum_{j=1}^n (\log \sigma(f_D(s_j^l, s_j^{l,o})) + \log \sigma(1 - f_D(s_j^l, \tilde{s}_j^l))) \quad (21)$$

where $s_j^{l,o}$ denotes student representation with counterfactual disruption. We make this contrastive learning mainly because contrasting the observed with the positive representation allows for learning a more accurate student representation, and comparing the observed with the negative sample allows for learning a more robust student representation.

F. Learning Objectives

1) Recommendation Loss: To make predictions for the target question q_t , we first combine its representation q_t with the student presentation s_N , and then map them through a neural network to a low-dimensional embedding, so as to calculate the probability that the student answers the target question correctly

$$p_{N+1} = \sigma(\mathbf{W}_3(\text{ReLU}(\mathbf{W}_2[s_N \oplus q_{N+1}] + \mathbf{b}_2)) + \mathbf{b}_3) \quad (22)$$

where $\mathbf{W}_2$, $\mathbf{W}_3$, $\mathbf{b}_2$, $\mathbf{b}_3$ are the model parameters, $\sigma(x)$ is the sigmoid function, $\oplus$ is the concatenation operation and p_{N+1} is the probability that the student answers the question correctly at Step $N+1$.

The cross-entropy loss is utilized to calculate the prediction loss by the above recommender (i.e., (22))

$$\mathcal{L}_q = - \sum_{t=1}^N (a_t \log p_t + (1 - a_t) \log(1 - p_t)). \quad (23)$$

2) Objective Function: To further consider the accuracy and robustness of students' representations, we then train the recommendation loss and the contrastive losses, jointly. That is, we achieve our final objective function by aggregating (23), (20), and (21)

$$\mathcal{L} = \mathcal{L}_q + \alpha \mathcal{L}_{cr} + \beta \mathcal{L}_{co} \quad (24)$$

where α and β are hyper-parameters to control the magnitude of the contrastive learning, $\mathcal{L}_{cr}$ and $\mathcal{L}_{co}$ denote the contrastive losses of counterfactual replacement and counterfactual disruption, respectively. $\mathcal{L}_{cr}$ and $\mathcal{L}_{co}$ are interpreted as maximizing the mutual information between the student representations learned in the two counterfactual transformation strategies. To learn the parameters, the gradient descent method is exploited. The computational complexity of our method mainly contains two parts. The first one is in finding the nearest neighbor question nodes, which needs to calculate the node-to-node distance to find the nearest neighbor node of the question node, and its computational cost is $O(m^2)$. The second one is in training the counterfactual learning model, which costs $O(LH^2m)$ to train the GCN encoder. Here L is the number of GCN layers, and H is the size of the node representation.

IV. EXPERIMENTAL SETTINGS

In this section, we first introduce our research questions and the three real-world datasets, and then describe the evaluation protocols, relevant baselines, and implementation details of our proposed approach.

A. Research Questions

We evaluate our proposal by answering the following research questions:

RQ1: How does our proposed GraphCA approach perform compared with other state-of-the-art KT methods?

RQ2: How do the student representations benefit from modeling the counterfactual world and contrastive learning?

RQ3: How do the hyper-parameters (i.e., M , N , and d) affect the performance of GraphCA?

RQ4: What are the training efficiency and scalability of GraphCA when processing large-scale data?

B. Datasets and Evaluation Protocols

To evaluate the performance of our proposed method, we use the following three real-world datasets. We report statistics for the ASSIST2009, ASSIST2012, and Algebra2006 datasets in Table I. For each dataset, we use 80% of the data as the training set and 20% of the data as the test set [3], [5].

ASSIST2009: This dataset is collected from the ASSISTments online education platform [65], which is widely used for KT. The education system can help students solve problems and improve their learning abilities. For this dataset, we remove records without concepts. This dataset has 3241 students, 17 709 questions, 124 concepts, and 278 868 records.

ASSIST2012: This dataset is collected from the same platform as ASSIST2009. The processing of the dataset is also in the same way as ASSIST2009. This dataset has 29 018 students, 53 086 questions, 265 concepts, and 2 711 602 records.

Bridge Algebra2006: Bridge to Algebra 2006–2007 stems from the KDD Cup 2010 EDM Challenge [66]. This dataset is based on an algebra course on the Cognitive Algebra Tutor System. This dataset has 1130 students, 129 263 questions, 550 concepts, and 1 817 393 records.

We adopt two commonly used evaluation metrics, area

TABLE I
STATISTICS OF THE DATASETS

Statistics	Datasets		
	ASSIST2009	ASSIST2012	Algebra2006
#Students	3241	29 018	1130
#Questions	17 709	53 086	129 263
#Concepts	124	265	550
#Records	278 868	2 711 602	1 817 393

under the curve (AUC) [31] and accuracy (ACC) [67], to measure the performance of the GraphCA. AUC is the area under the receiver operating characteristic (ROC) curve. ACC represents the error between the true value and the predicted value. When the value of AUC or ACC is 0.5, it indicates that the model performs as well as the random guess. A larger value of AUC or ACC indicates a better predictive performance of the model.

C. Baseline Methods

To validate the effectiveness of our model, we compare our GraphCA with the following baseline methods.

BKT [12]: This is a knowledge tracing model based on a Bayesian network, which exploits HMM to model students' knowledge state as a set of binary variables.

DKT [18]: This is the first knowledge tracing model based on a deep neural network, which uses RNN to model the learning situation of students. The input of DKT is the one-hot encoding of the student's knowledge representation and the output is a vector representing the degree of learner knowledge mastery.

DKVMN [19]: This typical model utilizes the idea of the memory-enhanced neural network, by using a static matrix key to store all knowledge concepts and a dynamic matrix value to store and update the student's knowledge state.

GKT [31]: This is a GNN-based knowledge tracing model which converts knowledge states into graphs for the prediction of student situations. At each time step, GKT updates the new state by aggregating the states of neighbors, and the states of neighbors are also updated.

SAKT [3]: This is a knowledge tracing model based on self-attention. SAKT accomplishes the prediction task by selecting more relevant KCs from the student's past activities.

AKT [68]: This is an attention-based network for knowledge tracing, and models learners' past performance by constructing context-aware representations for questions and responses.

GIKT [5]: This is a graph-based knowledge tracing interaction model that utilizes GCN to capture the rich relationships between questions and concepts. GIKT takes into account the interaction between the student's current state, the student's history states, the target question, and relevant concepts.

CLAKT [42]: This is a knowledge tracing model based on contrastive learning, which proposes four data augmentation methods thus capturing an effective knowledge state representation.

To better illustrate the characteristics and hierarchy of these models, we compare the different models in Tables II and III.

Among them, only the GIKT and our GraphCA model consider both direct and indirect relationships between concepts. For graph-based KT models GKT, GIKT, and GraphCA, GKT contains only concept nodes, i.e., the model considers only the relationship between concepts; GIKT contains both question and concept nodes, i.e., the model considers the relationship between questions and concepts and between concepts; in addition to the above two types of relationships, the GraphCA model takes into account the student-question relationship by introducing unified graphs involving students, questions, and concepts as multi-entities.

TABLE II
CHARACTERISTICS OF THE COMPARISON METHODS

	Modeling concept relations	Directed	Undirected
BKT	×	×	×
DKT	×	×	×
DKVMN	√	×	×
GKT	√	√	×
SAKT	√	√	×
AKT	√	√	×
GIKT	√	√	√
CL4KT	√	√	×
GraphCA	√	√	√

TABLE III
THE CONSIDERED ENTITY RELATIONSHIPS

	Concept-concept relations	Question-concept relations	Student-question relations
GKT	√	×	×
GIKT	√	√	×
GraphCA	√	√	√

D. Implementation Details

We implement GraphCA using the TensorFlow framework accelerated by NVIDIA RTX 2080 Ti GPU. By conducting extensive experiments, we obtain the optimal parameters for GraphCA. During the training process, we use the Adam algorithm [69] to optimize the parameters of our model, the learning rate is 0.001, and the mini-batch size is 64. For hyper-parameters, the number of replacement questions M is 5; the number of pairs of swap questions N is 5; the embedding sizes of students, questions, concepts, and answers are all set to $d = 100$. The details of tuning the hyper-parameters are shown in Section VI-B. We also use dropout [70] with the probability of 0.8 to prevent overfitting.

V. EXPERIMENTAL RESULTS (RQ1)

We compare the proposed GraphCA with existing methods on different datasets. We evaluate the performance of each model by measuring the AUC and ACC. As shown in Table IV, we report the results between baseline models and our GraphCA for the two classification metrics on the three datasets. In our evaluation phase, we run all the experiments ten times with the same parameter settings and report their

TABLE IV
COMPARISON RESULTS ON ALL THREE DATASETS IN TERMS OF AUC AND ACC

Method	ASSIST2009		ASSIST2012		Algebra2006	
	AUC	ACC	AUC	ACC	AUC	ACC
BKT	0.651 $\pm$ 0.001	0.594 $\pm$ 0.001	0.623 $\pm$ 0.003	0.602 $\pm$ 0.003	0.642 $\pm$ 0.001	0.656 $\pm$ 0.001
DKT	0.724 $\pm$ 0.004	0.716 $\pm$ 0.003	0.711 $\pm$ 0.001	0.697 $\pm$ 0.002	0.737 $\pm$ 0.005	0.834 $\pm$ 0.002
DKVMN	0.738 $\pm$ 0.004	0.718 $\pm$ 0.001	0.723 $\pm$ 0.001	0.733 $\pm$ 0.001	0.781 $\pm$ 0.001	0.833 $\pm$ 0.001
GKT	0.743 $\pm$ 0.002	0.704 $\pm$ 0.002	0.727 $\pm$ 0.003	0.710 $\pm$ 0.001	0.748 $\pm$ 0.001	0.835 $\pm$ 0.001
SAKT	0.750 $\pm$ 0.004	0.725 $\pm$ 0.002	0.745 $\pm$ 0.002	0.738 $\pm$ 0.001	0.767 $\pm$ 0.003	0.832 $\pm$ 0.002
AKT	0.783 $\pm$ 0.005	0.737 $\pm$ 0.002	0.772 $\pm$ 0.002	0.752 $\pm$ 0.002	0.737 $\pm$ 0.004	0.830 $\pm$ 0.001
GIKT	0.779 $\pm$ 0.004	0.734 $\pm$ 0.003	0.771 $\pm$ 0.002	0.746 $\pm$ 0.002	0.778 $\pm$ 0.002	0.834 $\pm$ 0.002
CL4KT	0.764 $\pm$ 0.003	0.722 $\pm$ 0.004	0.732 $\pm$ 0.006	0.731 $\pm$ 0.002	0.773 $\pm$ 0.003	0.798 $\pm$ 0.007
GraphCA	0.790 $\pm$ 0.003	0.741 $\pm$ 0.002	0.778 $\pm$ 0.001	0.748 $\pm$ 0.002	0.786 $\pm$ 0.003	0.835 $\pm$ 0.001

average values. To clearly show the significance of our improvements, all the comparison results also include their standard deviations. The best results for AUC and ACC on different datasets are shown in bold. As can be seen from the results, our model achieves high performance in most metrics on the three datasets. On the ASSIST2009 dataset, the AUC value of the GraphCA model is 79.0%, the AUC value of the GIKT model is 77.9%, and the AUC value of the AKT model is 78.3%. On the ASSIST2012 dataset, the AUC value of the GraphCA model is 77.8%, which is slightly better than the AKT and GIKT models. On the Algebra2006 dataset, the AUC value of the GraphCA model is 78.6%, the AUC value of the GIKT model is 77.8%, the AUC value of the CL4KT model is 77.3%, and the AUC value of the GKT model is 74.8%.

In all the baselines, the deep learning-based model has better performance than the traditional BKT model, which demonstrates the effectiveness of deep learning methods for predicting student performance. The SAKT model performs better than the DKT and DKVMN, which indicates that it is important to consider the relevance between the next question and the student's past activities. In addition, both models, GKT and GIKT, use the existing knowledge structure to obtain more additional useful information. From Table IV, we can find that the prediction results of GKT and GIKT are relatively better compared with other baselines. Furthermore, our model not only considers the relationship between questions and concepts but also includes the relationship between students and questions, which suggests that modeling multiple entities facilitate the predictive performance of the model.

VI. MODEL ANALYSIS

A. Ablation Study (RQ2)

To better understand the role of the different building blocks in the GraphCA, we conduct ablation experiments to illustrate the effect of each part of our model on the prediction results. We build the GraphCA-RCRO (Remove counterfactual transformation module for both views), GraphCA-RCR (Remove graph transformation module for counterfactual replacement), GraphCA-RCO (Remove graph transformation module for counterfactual disruption), and GraphCA-RCL (Remove con-

trastive learning) architectures by removing the counterfactual transformation module and contrastive learning module. The results of the four architectures on the ASSIST2009 dataset, ASSIST2012 dataset, and Algebra2006 dataset are shown in Table V.

TABLE V
ABLATION STUDIES BY CONSTRUCTING
DIFFERENT ARCHITECTURES

Method	ASSIST2009		ASSIST2012		Algebra2006	
	AUC	ACC	AUC	ACC	AUC	ACC
GraphCA-RCRO	0.770	0.731	0.765	0.743	0.767	0.831
GraphCA-RCR	0.779	0.734	0.770	0.739	0.769	0.830
GraphCA-RCO	0.780	0.735	0.771	0.741	0.776	0.830
GraphCA-RCL	0.784	0.739	0.770	0.747	0.777	0.835
GraphCA	0.790	0.741	0.778	0.748	0.786	0.835

GraphCA-RCRO: This suggests that neither counterfactual replacement of graph transformations nor counterfactual disruption of graph transformations are considered.

GraphCA-RCR: This suggests that the graph transformation process of counterfactual replacement is not considered.

GraphCA-RCO: This suggests that the graph transformation process of counterfactual disruption is not considered.

GraphCA-RCL: This suggests that the contrast between the observed and the counterfactual student representation is not considered.

The experimental results are shown in Table V, where the GraphCA model achieves the best performance when performing graph transformation from two views simultaneously. That is, GraphCA has a significant facilitation effect by combining two counterfactual transformation strategies. The above suggests that it is also critical to consider unobserved data when we track students' learning processes. From the results of GraphCA-RCR and GraphCA-RCO, we see that modeling counterfactual data from two views can improve the performance of the model. In addition, the better performance of GraphCA compared to GraphCA-RCL indicates that the presence of contrastive learning can learn more accurate and robust student representations. The prediction results of each

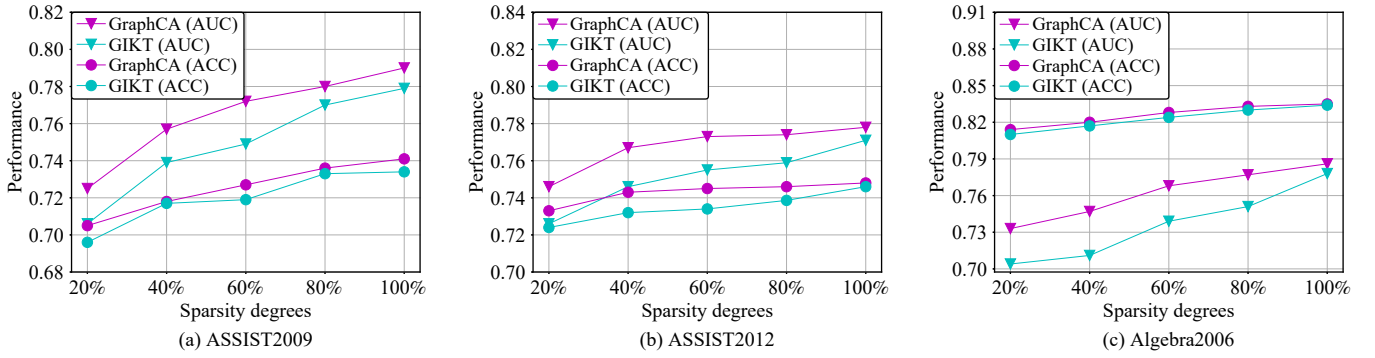


Fig. 4. The performance of GraphCA compared to the baseline under different sparsity degrees.

module answer RQ2 while demonstrating the effectiveness of our GraphCA.

To evaluate the performance of our method on sparse data, we further evaluate GraphCA by splitting the dataset into different sub-collections from 20% to 100%, and report the experimental results in Fig. 4, from which we can observe that our GraphCA method can perform better than the GIKT method (the most competitive baseline) on all three datasets. This result demonstrates the effectiveness of our counterfactual graph and the contrastive learning method in alleviating the data sparsity issue.

B. Hyper-Parameter Analysis (RQ3)

Our GraphCA model has three highly important hyper-parameters: the first is the number of replacement questions when performing the counterfactual transformation, the second is the number of pairs of swap questions when generating counterfactual data, and the last is the size of the embedding dimension. In this subsection, we will investigate the effect of different values of these three hyper-parameters on GraphCA.

1) *Impact of M and N* : We denote the number of replacement questions as M and the number of pairs of swap questions as N . We conduct experiments on different M and N , where M and N are chosen from $\{1, 5, 10\}$, to illustrate the impact of hyper-parameters on the prediction performance. From Table VI, we can find that the performance of GraphCA varies smoothly around the peak region, which is because the generated counterfactual data are positive samples and we do not change the semantic representation of the observed student to a great extent. Also, increasing the number of replacement questions and the number of pairs of swap questions may introduce noise because the selection of questions is randomly sampled. Although the performance of our model changes smoothly, we can still observe that the model performs relatively well when both M and N are 5.

2) *Impact of d* : We set the embedding dimensions d of student, question, and concept to be the same. The change in the performance of the model by changing the embedding dimension is illustrated in Fig. 5. We set d to $\{20, 40, 60, 80, 100\}$ to illustrate the effect of this hyper-parameter on the prediction performance. From Fig. 5, we can find that the prediction performance of our GraphCA becomes better and better as the embedding dimension increases. However, the values of AUC and ACC change very little when the dimensions reach a cer-

TABLE VI
IMPACT OF PARAMETERS M AND N

Number	ASSIST2009		ASSIST2012		Algebra2006	
	AUC	ACC	AUC	ACC	AUC	ACC
$M = 1, N = 1$	0.7868	0.7385	0.7778	0.7484	0.7768	0.8332
$M = 1, N = 5$	0.7871	0.7393	0.7774	0.7483	0.7820	0.8300
$M = 5, N = 5$	0.7902	0.7413	0.7783	0.7481	0.7867	0.8351
$M = 5, N = 1$	0.7874	0.7410	0.7777	0.7485	0.7773	0.8333
$M = 1, N = 10$	0.7869	0.7383	0.7767	0.7487	0.7827	0.8334
$M = 10, N = 1$	0.7878	0.7393	0.7770	0.7481	0.7835	0.8349
$M = 5, N = 10$	0.7879	0.7399	0.7780	0.7466	0.7835	0.8338
$M = 10, N = 5$	0.7886	0.7402	0.7778	0.7483	0.7821	0.8323

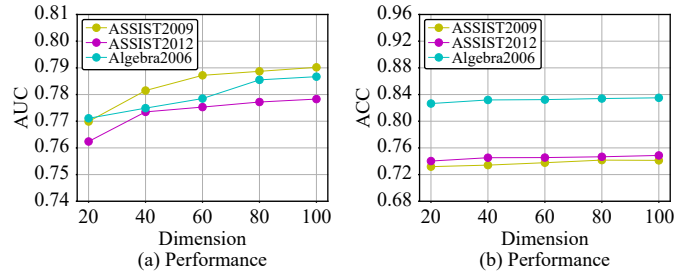


Fig. 5. Impact of parameter d .

tain number. In our experiments, we set $d = 100$.

C. Training Efficiency and Scalability (RQ4)

To investigate the training efficiency and scalability of our GraphCA, we measure the time required for different proportions of training data on three real-world datasets. Specifically, we vary the ratio of training data in $\{0.2, 0.4, 0.6, 0.8, 1.0\}$, and the detailed training time is shown in Fig. 6. Furthermore, we also include the linear training time related to the number of training samples to make our experimental results more comparable. The linear training time is calculated according to the linear relationship between the first two training times with linearly increasing the dataset split ratio from 20% to 100%. We term this time as the linear time as we thought the linear scalability of our method should increase linearly with the increase of the dataset ratio. From the experimental results in Fig. 6, we can observe that as the proportion of training data increases gradually from 0.2 to 1.0, the train-

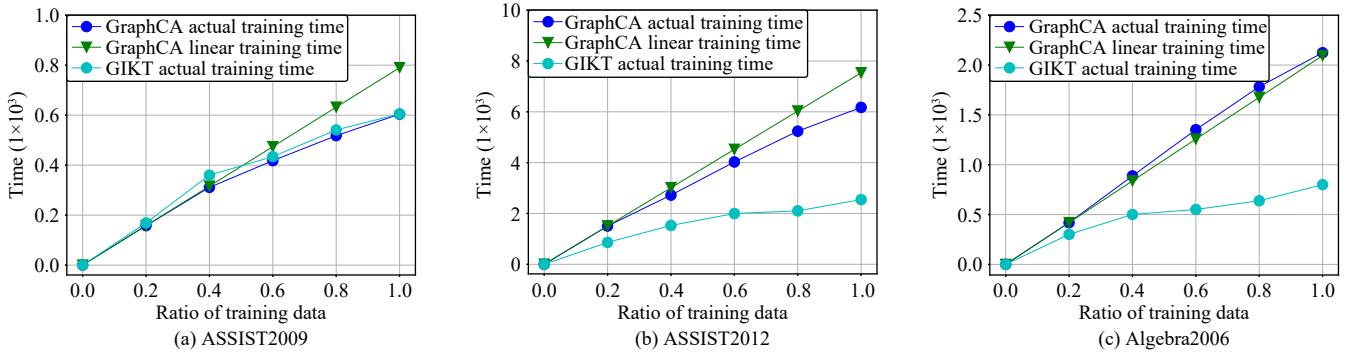


Fig. 6. The training time of GraphCA with different data ratios.

ing time cost of our GraphCA grows from 0.158×10^3 to 0.604×10^3 on the ASSIST2009 dataset, from 1.507×10^3 to 6.174×10^3 on the ASSIST2012 dataset, and from 0.419×10^3 to 2.124×10^3 on the Algebra2006 dataset. In addition, we can also observe that the actual training time of the model on the ASSIST2009 dataset and ASSIST2012 dataset is less than the linear training time, which demonstrates that the training time of the model is better than we expected. The overall trend on these three datasets indicates that the dependency of times cost for training GraphCA on the data scale is approximately linear. We further compare our method with GIKT (the most competitive method) on all three datasets on the basis of our previous scalability experiments. From the experimental results, we can find that though our method can scale well with the increase of the training data ratio, it has no advantage over the baseline method, due to its additional counterfactual learning cost.

D. Case Study

To make the counterfactual data modeling process of our model more intuitive and understandable, we provide a case study based on the ASSIST2009 dataset. In Fig. 7, we select an answer sequence with a student sequence length of 19. The first row is the student's observed answer records. The question IDs of the observed answer records are {15687, 15711, 15704, 15723, 15724, 15719, 15727, 13725, 14498, 13690, 17877, 17864, 14625, 14893, 14882, 14278, 14864, 14258, 14261}. We obtain the second and third rows of records by counterfactual transformation strategies. For the replacement question operation, we randomly select 5 questions from the observed answer records for replacement. The indexes of the replaced questions are {10, 6, 5, 4, 16}. By the similarity between the questions, we find the nearest neighbor to the replaced question among the unanswered questions. The IDs of the nearest neighboring questions are {1970, 3530, 16899, 1403, 3493}. For the third row, we only change the order of the questions, without changing the content of the answers. That is, the second and third rows do not fundamentally change the original student semantic representation, which suggests that the counterfactual transformation strategy mentioned in our model is reasonable. Fig. 8 depicts an example of a student's mastery of five knowledge concepts. From Fig. 8 we can find that the student's proficiency in the pythagorean theorem concept, median concept, and rounding concept

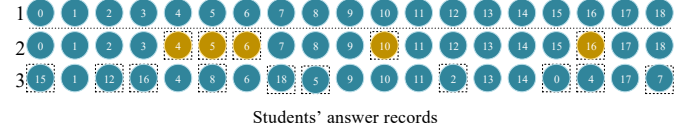


Fig. 7. A case study based on the ASSIST2009 dataset. In the first row, we show the observed student answer records, and the second and third rows are the records obtained by the counterfactual transformation of the two views.

increased, while proficiency in the circle graph concept decreased. After answering 19 questions, students mastered the pythagorean theorem concept, the median concept, and the rounding concept, but not the circle graph concept.

Additionally, we find that our method cannot do well for the students with only a few learning interactions. This is mainly because we represent students by their interacted questions in the graph, and when there are only a few records, these questions are not enough to accurately express a student's knowledge state. To show the limitation of our method, we randomly select a student with only 8 answering records (i.e., the length of the answer sequence is 8), and perform two counterfactual transformation strategies as shown in Section III-D. The student's knowledge state showing his mastery of knowledge concepts is presented in Fig. 9, from which we can observe that by answering fewer questions, we cannot capture the significant change in his/her mastery of the concepts.

VII. CONCLUSIONS AND FUTURE WORK

In this paper, we propose a knowledge tracing method based on GCN. We learn question representations and student representations by constructing the SQC graph. In addition, we perform counterfactual graph transformations from two views based on two factual observations. Then we design contrastive learning objectives to learn accurate and robust student representations. We conduct experiments on three real-world datasets, and the experimental results show that our proposed GraphCA model has better performance.

In the future, we would like to consider different strategies for counterfactual transformation (e.g., changing the number of original student answers) to implement the operations from multiple levels. Otherwise, there might be some more efficient way to obtain negative samples in the contrastive learning process. We also notice that some theories (e.g., learning ability) may affect the results of the KT.

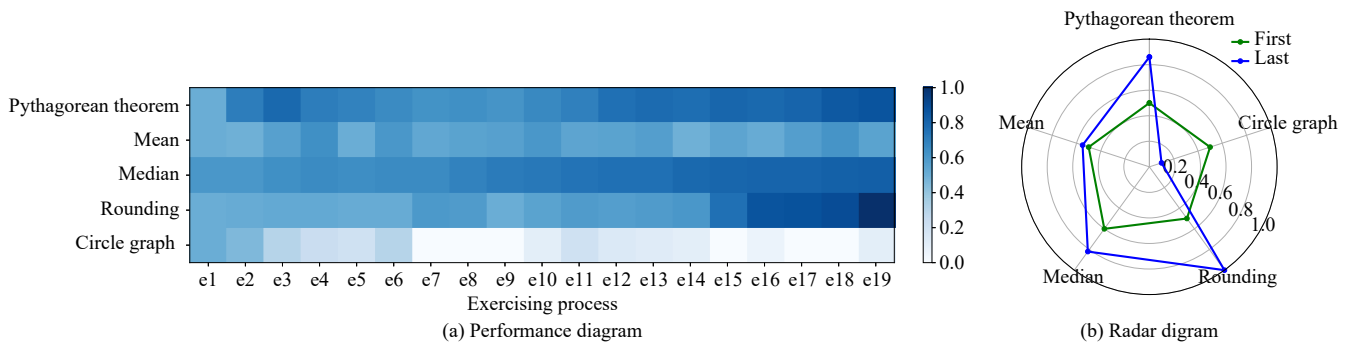


Fig. 8. An example of a student's mastery of five knowledge concepts.

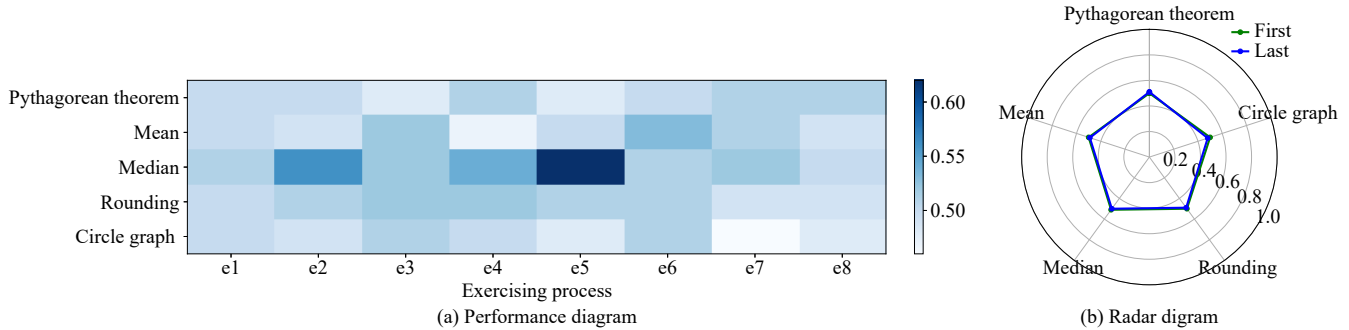


Fig. 9. The limitation of GraphCA.

REFERENCES

- [1] X. Wang, W. Ma, L. Guo, H. Jiang, F. Liu, and C. Xu, "HGNN: Hyperedge-based graph neural network for MOOC course recommendation," *Information Processing & Management*, vol. 59, no. 3, p. 102938, 2022.
- [2] C. Cui, J. Zong, Y. Ma, X. Wang, L. Guo, M. Chen, and Y. Yin, "Tri-branch convolutional neural networks for top-k focused academic performance prediction," *IEEE Trans. Neural Networks and Learning Systems*, 2022. DOI: 10.1109/TNNLS.2022.3175068
- [3] S. Pandey and G. Karypis, "A self-attentive model for knowledge tracing," arXiv preprint arXiv: 1907.06837, 2019.
- [4] T. Wang, F. Ma, and J. Gao, "Deep hierarchical knowledge tracing," in *Proc. 12th Int. Conf. Educational Data Mining*, 2019, pp. 671–674.
- [5] Y. Yang, J. Shen, Y. Qu, Y. Liu, K. Wang, Y. Zhu, W. Zhang, and Y. Yu, "GIKT: A graph-based interaction model for knowledge tracing," in *Proc. Joint European Conf. Machine Learning and Knowledge Discovery in Databases*, 2020, pp. 299–315.
- [6] P. Chen, Y. Lu, V. W. Zheng, and Y. Pian, "Prerequisite-driven deep knowledge tracing," in *Proc. IEEE Int. Conf. Data Mining*, 2018, pp. 39–48.
- [7] S. Zhang, D. Yao, Z. Zhao, T.-S. Chua, and F. Wu, "CauseRec: Counterfactual user sequence synthesis for sequential recommendation," in *Proc. 44th Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, 2021, pp. 367–377.
- [8] L. V. DiBello, L. A. Roussos, and W. Stout, "31A review of cognitively diagnostic assessment and a summary of psychometric models," *Handbook of Statistics*, vol. 26, pp. 979–1030, 2006.
- [9] R. J. Harvey and A. L. Hammer, "Item response theory," *The Counseling Psychologist*, vol. 27, no. 3, pp. 353–383, 1999.
- [10] B. Deonovic, M. Yudelson, M. Bolsinova, M. Attali, and G. Maris, "Learning meets assessment," *Behaviormetrika*, vol. 45, no. 2, pp. 457–474, 2018.
- [11] J. De La Torre, "Dina model and parameter estimation: A didactic," *J. Educational Behavioral Statistics*, vol. 34, no. 1, pp. 115–130, 2009.
- [12] A. T. Corbett and J. R. Anderson, "Knowledge tracing: Modeling the acquisition of procedural knowledge," *User Modeling and User-Adapted Interaction*, vol. 4, no. 4, pp. 253–278, 1994.
- [13] R. S. d Baker, A. T. Corbett, and V. Aleven, "More accurate student modeling through contextual estimation of slip and guess probabilities in bayesian knowledge tracing," in *Proc. Int. Conf. Intelligent Tutoring Systems*, 2008, pp. 406–415.
- [14] S. Spaulding and C. Breazeal, "Affect and inference in bayesian knowledge tracing with a robot tutor," in *Proc. 10th Annu. ACM/IEEE Int. Conf. Human-Robot Interaction Extended Abstracts*, 2015, pp. 219–220.
- [15] Z. A. Pardos, N. Heffernan, C. Ruiz, and J. Beck, "Effective skill assessment using expectation maximization in a multi network temporal bayesian network," in *Proc. Young Researchers Track at 9th Int. Conf. Intelligent Tutoring Systems*, 2008, pp. 1–14.
- [16] L. Guo, H. Yin, Q. Wang, T. Chen, A. Zhou, and N. Quoc Viet Hung, "Streaming session-based recommendation," in *Proc. 25th ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining*, 2019, pp. 1569–1577.
- [17] L. Guo, J. Zhang, T. Chen, X. Wang, and H. Yin, "Reinforcement learning-enhanced shared-account cross-domain sequential recommendation," *IEEE Trans. Knowledge and Data Engineering*, vol. 35, no. 7, pp. 7397–7411, Jul. 2023.
- [18] C. Piech, J. Bassen, J. Huang, S. Ganguli, M. Sahami, L. J. Guibas, and J. Sohl-Dickstein, "Deep knowledge tracing," *Advances in Neural Information Processing Syst.*, vol. 1, pp. 505–513, 2015.
- [19] J. Zhang, X. Shi, I. King, and D.-Y. Yeung, "Dynamic key-value memory networks for knowledge tracing," in *Proc. 26th Int. Conf. World Wide Web*, 2017, pp. 765–774.
- [20] K. Nagatani, Q. Zhang, M. Sato, Y.-Y. Chen, F. Chen, and T. Ohkuma, "Augmenting knowledge tracing by considering forgetting behavior," in *Proc. World Wide Web Conf.*, 2019, pp. 3101–3107.
- [21] G. Abdelrahman and Q. Wang, "Knowledge tracing with sequential key-value memory networks," in *Proc. 42nd Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, 2019, pp. 175–184.
- [22] Q. Liu, Z. Huang, Y. Yin, E. Chen, H. Xiong, Y. Su, and G. Hu, "EKT: Exercise-aware knowledge tracing for student performance prediction," *IEEE Trans. Knowledge and Data Engineering*, vol. 33, no. 1, pp. 100–115, 2019.
- [23] L. Guo, L. Tang, T. Chen, L. Zhu, Q. V. H. Nguyen, and H. Yin,

- “DAGCN: A domain-aware attentive graph convolution network for sharedaccount cross-domain sequential recommendation,” arXiv preprint arXiv: 2105.03300, 2021.
- [24] L. Guo, H. Yin, T. Chen, X. Zhang, and K. Zheng, “Hierarchical hyperedge embedding-based representation learning for group recommendation,” *ACM Trans. Information Systems*, vol. 40, no. 1, pp. 1–27, 2021.
- [25] X. Wang, L. Jia, L. Guo, and F. Liu, “Multi-aspect heterogeneous information network for MOOC knowledge concept recommendation,” *Applied Intelligence*, vol. 53, pp. 11951–11965, 2023.
- [26] X. Hong, T. Zhang, Z. Cui, and J. Yang, “Variational gridded graph convolution network for node classification,” *IEEE/CAA J. Autom. Sinica*, vol. 8, no. 10, pp. 1697–1708, 2021.
- [27] X. Wang, X. He, Y. Cao, M. Liu, and T.-S. Chua, “KGAT: Knowledge graph attention network for recommendation,” in *Proc. 25th ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining*, 2019, pp. 950–958.
- [28] X. Qi, R. Liao, J. Jia, S. Fidler, and R. Urtasun, “3D graph neural networks for RGBD semantic segmentation,” in *Proc. IEEE Int. Conf. Computer Vision*, 2017, pp. 5199–5208.
- [29] L. Yao, C. Mao, and Y. Luo, “Graph convolutional networks for text classification,” in *Proc. AAAI Conf. Artificial Intelligence*, 2019, vol. 33, no. 1, pp. 7370–7377.
- [30] Y. Liu, Y. Yang, X. Chen, J. Shen, H. Zhang, and Y. Yu, “Improving knowledge tracing via pre-training question embeddings,” arXiv preprint arXiv: 2012.05031, 2020.
- [31] H. Nakagawa, Y. Iwasawa, and Y. Matsuo, “Graph-based knowledge tracing: Modeling student proficiency using graph neural network,” in *Proc. IEEE/WIC/ACM Int. Conf. Web Intelligence*, 2019, pp. 156–163.
- [32] S. Tong, Q. Liu, W. Huang, Z. Huang, E. Chen, C. Liu, H. Ma, and S. Wang, “Structure-based knowledge tracing: An influence propagation view,” in *Proc. IEEE Int. Conf. Data Mining*, 2020, pp. 541–550.
- [33] K. Clark, M.-T. Luong, Q. V. Le, and C. D. Manning, “ELECTRA: Pretraining text encoders as discriminators rather than generators,” arXiv preprint arXiv: 2003.10555, 2020.
- [34] T. Gao, X. Yao, and D. Chen, “SimCSE: Simple contrastive learning of sentence embeddings,” arXiv preprint arXiv: 2104.08821, 2021.
- [35] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, “Momentum contrast for unsupervised visual representation learning,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition*, 2020, pp. 9729–9738.
- [36] H. Hu, H. Wang, Z. Liu, and W. Chen, “Domain-invariant similarity activation map contrastive learning for retrieval-based long-term visual localization,” *IEEE/CAA J. Autom. Sinica*, vol. 9, no. 2, pp. 313–328, 2021.
- [37] X. Xie, F. Sun, Z. Liu, S. Wu, J. Gao, B. Ding, and B. Cui, “Contrastive learning for sequential recommendation,” arXiv preprint arXiv: 2010.14395, 2020.
- [38] T. Yao, X. Yi, D. Z. Cheng, *et al.*, “Self-supervised learning for large-scale item recommendations,” in *Proc. 30th ACM Int. Conf. Information & Knowledge Management*, 2021, pp. 4321–4330.
- [39] K. Hassani and A. H. Khasahmadi, “Contrastive multi-view representation learning on graphs,” in *Proc. Int. Conf. Machine Learning*, 2020, pp. 4116–4126.
- [40] H. Hafidi, M. Ghogho, P. Ciblat, and A. Swami, “GraphCL: Contrastive self-supervised learning of graph representations,” arXiv preprint arXiv: 2007.08025, 2020.
- [41] X. Song, J. Li, Q. Lei, W. Zhao, Y. Chen, and A. Mian, “Bi-CLKT: Bi-graph contrastive learning based knowledge tracing,” arXiv preprint arXiv: 2201.09020, 2022.
- [42] W. Lee, J. Chun, Y. Lee, K. Park, and S. Park, “Contrastive learning for knowledge tracing,” in *Proc. ACM Web Conf.*, 2022, pp. 2330–2338.
- [43] C. Zhang, D. Song, C. Huang, A. Swami, and N. V. Chawla, “Heterogeneous graph neural network,” in *Proc. 25th ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining*, 2019, pp. 793–803.
- [44] X. Wang, H. Ji, C. Shi, B. Wang, Y. Ye, P. Cui, and P. S. Yu, “Heterogeneous graph attention network,” in *Proc. World Wide Web Conf.*, 2019, pp. 2022–2032.
- [45] S. Xu, C. Yang, C. Shi, Y. Fang, Y. Guo, T. Yang, L. Zhang, and M. Hu, “Topic-aware heterogeneous graph neural network for link prediction,” in *Proc. 30th ACM Int. Conf. Information & Knowledge Management*, 2021, pp. 2261–2270.
- [46] Y. Sun, J. Han, C. C. Aggarwal, and N. V. Chawla, “When will it happen? Relationship prediction in heterogeneous information networks,” in *Proc. 5th Conf. Web Search and Data Mining*, 2012, pp. 663–672.
- [47] J. Shi, H. Ji, C. Shi, X. Wang, Z. Zhang, and J. Zhou, “Heterogeneous graph neural network for recommendation,” arXiv preprint arXiv: 2009.00799, 2020.
- [48] S. Liu, I. Ounis, C. Macdonald, and Z. Meng, “A heterogeneous graph neural model for cold-start recommendation,” in *Proc. 43rd Int. ACM SIGIR Conf. Research and Development in Infor. Retrieval*, 2020, pp. 2029–2032.
- [49] J. Zhao, X. Wang, C. Shi, B. Hu, G. Song, and Y. Ye, “Heterogeneous graph structure learning for graph neural networks,” in *Proc. AAAI Conf. Artificial Intelligence*, 2021, vol. 35, no. 5, pp. 4697–4705.
- [50] S. Garg, V. Perot, N. Limtiaco, A. Taly, E. H. Chi, and A. Beutel, “Counterfactual fairness in text classification through robustness,” in *Proc. AAAI/ACM Conf. AI, Ethics, and Society*, 2019, pp. 219–226.
- [51] M. J. Kusner, J. Loftus, C. Russell, and R. Silva, “Counterfactual fairness,” *Advances in Neural Information Processing Systems*, vol. 30, pp. 4069–4079, 2017.
- [52] Z. Wang, J. Zhang, H. Xu, X. Chen, Y. Zhang, W. X. Zhao, and J.-R. Wen, “Counterfactual data-augmented sequential recommendation,” in *Proc. 44th Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, 2021, pp. 347–356.
- [53] R. Zmigrod, S. J. Mielke, H. Wallach, and R. Cotterell, “Counterfactual data augmentation for mitigating gender stereotypes in languages with rich morphology,” arXiv preprint arXiv: 1906.04571, 2019.
- [54] T.-J. Fu, X. E. Wang, M. F. Peterson, S. T. Grafton, M. P. Eckstein, and W. Y. Wang, “Counterfactual vision-and-language navigation via adversarial path sampler,” in *Proc. European Conf. Computer Vision*, 2020, pp. 71–86.
- [55] E. Abbasnejad, D. Teney, A. Parvaneh, J. Shi, and A. v. d. Hengel, “Counterfactual vision and language learning,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition*, 2020, pp. 10044–10054.
- [56] L. Chen, X. Yan, J. Xiao, H. Zhang, S. Pu, and Y. Zhuang, “Counterfactual samples synthesizing for robust visual question answering,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition*, 2020, pp. 10800–10809.
- [57] H. Wang, W. Liang, J. Shen, L. Van Gool, and W. Wang, “Counterfactual cycle-consistent learning for instruction following and generation in vision-language navigation,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition*, 2022, pp. 15471–15481.
- [58] Y. Zhu, Y. Xu, F. Yu, Q. Liu, S. Wu, and L. Wang, “Graph contrastive learning with adaptive augmentation,” in *Proc. Web Conf.*, 2021, pp. 2069–2080.
- [59] Y. Jiao, Y. Xiong, J. Zhang, Y. Zhang, T. Zhang, and Y. Zhu, “Sub-graph contrast for scalable self-supervised graph representation learning,” in *Proc. IEEE Int. Conf. Data Mining*, 2020, pp. 222–231.
- [60] Y. You, T. Chen, Y. Sui, T. Chen, Z. Wang, and Y. Shen, “Graph contrastive learning with augmentations,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 5812–5823, 2020.
- [61] T. Zhao, G. Liu, D. Wang, W. Yu, and M. Jiang, “Learning from counterfactual links for link prediction,” in *Proc. Int. Conf. Machine Learning*, 2022, pp. 26911–26926.
- [62] G. W. Imbens and D. B. Rubin, *Causal Inference in Statistics, Social, and Biomedical Sciences*. Cambridge, UK: Cambridge University Press, 2015.
- [63] P. C. Austin, “An introduction to propensity score methods for reducing the effects of confounding in observational studies,” *Multivariate Behavioral Research*, vol. 46, no. 3, pp. 399–424, 2011.
- [64] A. Van den Oord, Y. Li, and O. Vinyals, “Representation learning with contrastive predictive coding,” arXiv preprint arXiv: 1807.03748, 2018.

- [65] M. Feng, N. Heffernan, and K. Koedinger, “Addressing the assessment challenge with an online system that tutors as it assesses,” *User Modeling and User-Adapted Interaction*, vol. 19, no. 3, pp. 243–266, 2009.
- [66] J. Stamper, A. Niculescu-Mizil, S. Ritter, G. Gordon, and K. Koedinger, “Algebra I 2005–2006 and bridge to algebra 2006–2007 development data sets from KDD cup 2010 educational data mining challenge,” 2010. [Online], Available: <https://pslcdatashop.web.cmu.edu/KDDCup/downloads.jsp>.
- [67] M. Zhang, X. Zhu, C. Zhang, Y. Ji, F. Pan, and C. Yin, “Multifactors aware dual-attentional knowledge tracing,” in *Proc. 30th ACM Int. Conf. Information & Knowledge Management*, 2021, pp. 2588–2597.
- [68] A. Ghosh, N. Heffernan, and A. S. Lan, “Context-aware attentive knowledge tracing,” in *Proc. 26th ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining*, 2020, pp. 2330–2339.
- [69] D. P. Kingma and J. Ba, “ADAM: A method for stochastic optimization,” arXiv preprint arXiv: 1412.6980, 2014.
- [70] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *The J. Machine Learning Research*, vol. 15, no. 1, pp. 1929–1958, 2014.



Xinhua Wang received the Ph.D. degree in management science and engineering from Shandong Normal University in 2008. He was a Senior Visiting Scholar with Shandong University, from 2002 to 2003. He was a Senior Visiting Scholar with Peking University, from 2008 to 2009. He is a Professor and Master’s Supervisor at the School of Information Science and Engineering, Shandong Normal University. His research interests include data mining and recommender systems.

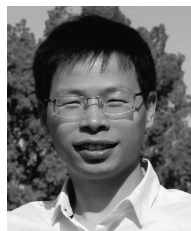


Shasha Zhao is a master student in computer science and technology at the School of Information Science and Engineering, Shandong Normal University. Her research interests include sequence modeling and education data mining.



Lei Guo received the Ph.D. degree in computer science from Shandong University in 2015. He was a Visiting Scholar at The University of Queensland, Australia, from 2018 to 2019. He is an Associate Professor and a Master Supervisor at the School of Information Science and Engineering, Shandong Normal University. His research interests include data mining, recommender systems, and social media mining.

He is a Member of the Social Media Processing Committee of the Chinese Information Society.



Lei Zhu received the Ph.D. degree in computer science from Huazhong University Science in 2015. He is a Professor and Doctoral Supervisor with the School of Information Science and Engineering, Shandong Normal University. His research interests include multimedia computing, information retrieval and data mining.

He serves as an Editorial Board Member of journals *IEEE TBD*, *ACM TOMM*, and *Information Sciences*, chairs the field of ACM MM, an authoritative international conference in multimedia, and is a Member of the senior program committee of CIKM, AAAI, and SIGIR.



Chaoran Cui received the Ph.D. degree in computer science from Shandong University in 2015. He is a Professor at Shandong University of Finance and Economics. His research interests include information retrieval, data mining, and machine learning.

He is the Executive Director of the Shandong Artificial Intelligence Society, a Member of the Artificial Intelligence and Pattern Recognition Committee of the Chinese Computer Society, and a Member of the Computer Vision Committee of the Chinese Computer Society.



Liancheng Xu received the Ph.D. degree from the Dalian University of Technology in 2010. He is currently a Senior Experimentalist and Director of the Experimental Teaching Center, School of Information Science and Engineering, Shandong Normal University and a Master Supervisor. His research interests include web information processing and education data mining.

He is a Member of the Chinese Computer Society and ACM.